



# Security Assessment

## Final Report



# Polymarket V2

## Combinatorial Module

May 2026

Prepared for Polymarket

# Table of Contents

|                                                                                                                    |           |
|--------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Project Summary</b>                                                                                             | <b>3</b>  |
| Project Scope                                                                                                      | 3         |
| Project Overview                                                                                                   | 3         |
| Protocol Overview                                                                                                  | 3         |
| <b>Assessment Methodology</b>                                                                                      | <b>5</b>  |
| <b>Threat and Security Overview</b>                                                                                | <b>6</b>  |
| System Level Model & Security Relevant Elements                                                                    | 6         |
| Expected Invariants, Trust Boundaries, & Security Assumptions                                                      | 6         |
| Attack Vectors Considered                                                                                          | 7         |
| Findings Summary                                                                                                   | 8         |
| Severity Matrix                                                                                                    | 8         |
| <b>Detailed Findings</b>                                                                                           | <b>9</b>  |
| High Severity Issues                                                                                               | 10        |
| H-01: Main protocol invariant can be broken by an attacker                                                         | 10        |
| Medium Severity Issues                                                                                             | 13        |
| M-01: Redeems require all markets to be resolved causing temporary DoS                                             | 13        |
| M-02: Combinatorial positions across chains do not work as intended                                                | 15        |
| Low Severity Issues                                                                                                | 17        |
| L-01: Compressing will not work correctly in an edge case                                                          | 17        |
| L-02: Order dependence in <code>_getPositionPayout</code> prevents early settlement of combinatorial YES positions | 19        |
| Informational Severity Issues                                                                                      | 21        |
| I-01: <code>prepareCondition</code> should use <code>getConditionId</code> to compute the <code>conditionId</code> | 21        |
| I-02: Array trimming functionality can be made into an $O(1)$ operation                                            | 22        |
| <b>Disclaimer</b>                                                                                                  | <b>24</b> |
| <b>About Certora</b>                                                                                               | <b>24</b> |

# Project Summary

## Project Scope

| Project Name                       | Repository Link                                                                                                                                                           | Commit Hash                                                        | Platform |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|----------|
| Polymarket V2 Combinatorial Module | <a href="https://github.com/Polymarket/polymarket-v2/tree/release/combinatorial-module">https://github.com/Polymarket/polymarket-v2/tree/release/combinatorial-module</a> | <a href="#">f6585d3</a> (initial)<br><a href="#">abedba3</a> (fix) | Solidity |

## Project Overview

This document describes the security review of **Polymarket V2 Combinatorial Module**. The work was undertaken from **April 27th, 2026** to **May 4th, 2026**.

The following contract list is included in our scope:

```
src/modules/CombinatorialModule.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

## Protocol Overview

Polymarket V2 is a modular prediction market protocol built on Solidity 0.8.34 with Foundry and Solady. It manages prediction market positions as ERC-1155 tokens through a central **PositionManager** that routes to pluggable market modules (**BinaryModule** and **NegRiskModule**).

Positions use structured IDs that encode all routing information directly in the token ID (**moduleId**, **baseHash**, **conditionIndex**, **outcomeIndex**), eliminating storage lookups for routing. An **OracleAggregator** provides a modular oracle framework with pluggable reporter, disputer, and arbitrator modules. An **Exchange** contract handles EIP-712 signed order matching for position trading. A **BridgeBase** abstract contract enables cross-chain position and collateral transfers.

Core operations (split, merge, redeem) follow an effects-first callback pattern: the module mints output tokens, calls back to the sender to pull input tokens, then verifies the input arrived. The **Router** uses a zero-callback variant where assets are pre-transferred before calling the module. This callback pattern intentionally enables flash-loan-like behavior — callers receive minted tokens before paying for them.

This review focused only on the **CombinatorialModule** which is an addition to the currently existing **BinaryModule** and **NegRiskModule**.

The **CombinatorialModule** (moduleId=3) enables multi-leg parlay markets by managing YES combinatorial tokens that represent the strict conjunction (AND) of multiple underlying binary or negative risk conditions, alongside NO tokens representing their mathematical complement. It supports highly dynamic position management – including splitting, merging, condition extraction/injection, and token wrapping – while allowing users to incrementally compress partially resolved parlays and eventually redeem resolved winning positions for collateral.

# Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

# Threat and Security Overview

## System Level Model & Security Relevant Elements

The `CombinatorialModule` (moduleId=3) acts as the parlay execution engine for Polymarket V2. It is an ERC-1155 token receiver and an `IPositionManagerModule` that interfaces directly with the central `PositionManager` (for minting/burning position tokens) and the `CollateralToken` (for holding backing assets).

The system models a YES combinatorial token as the strict conjunction (AND) of multiple underlying Binary or Negative Risk conditions, while the NO token acts as its mathematical complement (OR). Security-relevant elements include the canonical sorting engine (`_validateCanonical`), which prevents duplicate or conflicting conditions, the strict overcollateralization that needs to be ensured at all times, meaning no user should profit from split/merge/extract/compress/redeem, correct bridging of positions, and the newly implemented `crossModuleAuth` flag.

## Expected Invariants, Trust Boundaries, & Security Assumptions

### Invariants:

- **Collateral Backing:** The combined payout of a YES token and its complementary NO token must always exactly equal the total split/merge collateral amount (`yesPayout + noPayout == amount`).
- **Canonical State:** A combinatorial condition Id must strictly map to an ascending, duplicate-free array of valid underlying conditions to prevent cyclic routing or double-counting.

### Trust Boundaries:

- **Owner/Admin:** The `onlyOwner` modifier protects UUPS upgrades (`_authorizeUpgrade`), while `onlyAdmin` controls bridge permissions.

- **Bridges:** Addresses holding the bridge role (`_ROLE_3`) are trusted to `mintFromBridge` and `burnFromBridge` without providing underlying collateral.

### Security Assumptions:

- The module assumes that underlying modules (Binary/NegRisk) will strictly return payout values bounded between `0` and `RESULT_DENOMINATOR (1_000_000)`.

### Attack Vectors Considered

- **Economic Risks:** Exploiting fixed-point math and rounding truncation in partial resolutions to break the core `1 YES + 1 NO = 1$` invariant, forcing the system to overpay and become undercollateralized. (H-01)
- **Cross-Protocol Interactions:** Cross-chain liquidity fragmentation where bridged combinatorial positions are permanently bricked and un-preparable on destination chains due to the inclusion of `block.chainid`. (M-02)
- **Lifecycle Denial of Service:** Redeems requiring all legs to be resolved, causing temporary DoS for users whose positions already have a mathematically guaranteed payout due to a fully failed leg. (M-01)
- **Splitting/merging non-existent positions:** Due to missing checks in split and merge functions anyone can prepare non-existent positions.
- **Preparing an invalid combinatorial condition:** An invalid combinatorial condition can be prepared as `prepareCondition` is permissionless and the conditions that are included can be invalid. For example anyone can include a position in which the arity encoded is higher than the actual conditions count.
- **Abusing the `crossModuleAuth` flag:** Because the `CombinatorialModule` will also be able to mint positions for other modules, it might be possible to mint positions that have an invalid module.

## Findings Summary

The table below summarizes the findings of the review, including type and severity details.

| Severity      | Discovered | Confirmed | Fixed    |
|---------------|------------|-----------|----------|
| Critical      | –          | –         | –        |
| High          | 1          | 1         | 1        |
| Medium        | 2          | 2         | 2        |
| Low           | 2          | 2         | 2        |
| Informational | 2          | 2         | 1        |
| <b>Total</b>  | <b>7</b>   | <b>7</b>  | <b>6</b> |

## Severity Matrix

| Impact     | High   | Medium | High   | Critical |
|------------|--------|--------|--------|----------|
|            | Medium | Low    | Medium | High     |
|            | Low    | Low    | Low    | Medium   |
|            |        | Low    | Medium | High     |
| Likelihood |        |        |        |          |



# Detailed Findings

| ID                   | Title                                                                                                        | Severity      | Status       |
|----------------------|--------------------------------------------------------------------------------------------------------------|---------------|--------------|
| <a href="#">H-01</a> | Main protocol invariant can be broken by an attacker                                                         | High          | Fixed        |
| <a href="#">M-01</a> | Redeems require all markets to be resolved causing temporary DoS                                             | Medium        | Fixed        |
| <a href="#">M-02</a> | Combinatorial positions across chains do not work as intended                                                | Medium        | Fixed        |
| <a href="#">L-01</a> | Compressing will not work correctly in an edge case                                                          | Low           | Fixed        |
| <a href="#">L-02</a> | Order dependence in <code>_getPositionPayout</code> prevents early settlement of combinatorial YES positions | Low           | Fixed        |
| <a href="#">I-01</a> | <code>prepareCondition</code> should use <code>getConditionId</code> to compute the <code>conditionId</code> | Informational | Acknowledged |
| <a href="#">I-02</a> | Array trimming functionality can be made into an $O(1)$ operation                                            | Informational | Fixed        |

## High Severity Issues

### H-01: Main protocol invariant can be broken by an attacker

|                                               |               |                    |
|-----------------------------------------------|---------------|--------------------|
| Severity: High                                | Impact: High  | Likelihood: Medium |
| Files:<br>src/modules/CombinatorialModule.sol | Status: Fixed |                    |

#### Description:

The payouts for a parlay are computed as:

```
for (uint256 i; i < length;) {
    (bool resolved, uint256 conditionPayout) =
    _getConditionPayout(storedConditions[i]);
    require(resolved, PositionNotRedeemable());
    yesPayout = FixedPointMathLib.mulDiv(yesPayout, conditionPayout,
    RESULT_DENOMINATOR);
    unchecked {
        ++i;
    }
}

return outcomeIndex == 0 ? yesPayout : _amount - yesPayout;
```

As seen by the above code, there is a rounding-up behaviour for the NO side as it is computed by subtracting a rounded-down value.

This can be abused by an attacker to force the system to pay out more tokens than he provided, effectively breaking the protocol-wide  $1 \text{ YES} + 1 \text{ NO} = 1\$$  invariant and making the system undercollateralized. The attack looks as follows:

1. There are 3 markets that have been resolved to 0.5 – A, B and C.
2. Attacker starts with 369\$ (calculations assume 1-decimal for the collateral token for simplicity).

3. Attacker splits the following, spending 123\$ on each:

- $\neg A$  condition, get  $\text{YES}(\neg A) + \text{NO}(\neg A)$
- $A \wedge \neg B$  condition, get  $\text{YES}(A \wedge \neg B) + \text{NO}(A \wedge \neg B)$
- $A \wedge B \wedge \neg C$  condition, get  $\text{YES}(A \wedge B \wedge \neg C) + \text{NO}(A \wedge B \wedge \neg C)$

4. Use `mergeFromYesBasket()` to turn the YES side of the splits into  $\text{NO}(A \wedge B \wedge C)$ .

5. Now the attacker has the below balances, note how each position is on the NO-side:

- $\text{NO}(A \wedge B \wedge C) - 123$
- $\text{NO}(\neg A) - 123$
- $\text{NO}(A \wedge \neg B) - 123$
- $\text{NO}(A \wedge B \wedge \neg C) - 123$

6. Attacker redeems the positions:

- $\text{NO}(A \wedge B \wedge C) = 123 - 15 = 108$
- $\text{NO}(\neg A) = 123 - 61 = 62$
- $\text{NO}(A \wedge \neg B) = 123 - 30 = 93$
- $\text{NO}(A \wedge B \wedge \neg C) = 123 - 15 = 108$

7. The total being  $108 + 62 + 93 + 108 = 371$ . The attacker started with 369, a profit of 2. The attack is repeatable and immediate.

A simpler attack:

1. Split a 1-leg parlay based on a position that has resolved 50:50.
2. Redeem the NO side providing 1 wei on each iteration which implicitly rounds up and gives a payout of 1 wei. Repeat for the whole amount we split.
3. Redeem the YES side in 1 iteration.
4. Outcome is that the user extracted 50% more than the value he split in position 1 as the NO redeems overpays him on each iteration.

### Recommendations:

Consider computing the NO-side independently and not via subtraction from the YES-side.

### Customer Response:

Fixed in commits [2959144](#) and [2119277](#).



**Fix Review:**

Fix confirmed.

## Medium Severity Issues

### M-01: Redeems require all markets to be resolved causing temporary DoS

| Severity: Medium                              | Impact: Medium | Likelihood: Medium |
|-----------------------------------------------|----------------|--------------------|
| Files:<br>src/modules/CombinatorialModule.sol | Status: Fixed  |                    |

#### Description:

By definition, a parlay position pays out 0 even if a single leg resolves not in favour of the YES holder and in that case, the NO holder gets a payout of 1. Thus, a correct behaviour should be that even if all other markets in the parlay are unresolved, the position should be redeemable as the outcome is already known. However, the current implementation requires all legs to be resolved:

```
for (uint256 i; i < length;) {
    (bool resolved, uint256 conditionPayout) =
    _getConditionPayout(storedConditions[i]);
    require(resolved, PositionNotRedeemable());
    yesPayout = FixedPointMathLib.mulDiv(yesPayout, conditionPayout,
    RESULT_DENOMINATOR);
    unchecked {
        ++i;
    }
}
```

This is problematic as a user might hold a NO position of 2 events – one resolving today, one in a few years. If the one resolving today resolves in his favour (leg is **A**, thus resolves **NOT(A)**), then the outcome is already known (he should get the full payout), but will have to wait years for the **B** leg to resolve and get his payout.

#### Recommendations:

Implement logic that handles this case – if **yesPayout** is 0 on any given leg, the outcome is known.

**Customer Response:**

Fixed in commit [2119277](#).

**Fix Review:**

Fix confirmed.

## M-02: Combinatorial positions across chains do not work as intended

|                                               |                |                    |
|-----------------------------------------------|----------------|--------------------|
| Severity: Medium                              | Impact: Medium | Likelihood: Medium |
| Files:<br>src/modules/CombinatorialModule.sol | Status: Fixed  |                    |

### Description:

Users can bridge combinatorial positions across chains. It is expected that they can do different operations on spoke chains such as redeeming, compressing, injecting, etc., however the issue makes all of these impossible.

Whenever a position is bridged, `CombinatorialModule::mintFromBridge()` is called which simply mints the position as the only state change. The issue is that the position's condition can never be prepared as that happens via `prepareCondition()` which includes the `block.chainid` when computing the condition:

```
conditionId = PositionIdLib.encodeConditionIdFromData(block.chainid,  
ModuleIds.COMBINATORIAL, 0, abi.encode(_conditions));
```

This means that the same underlying conditions would have a different condition on each chain, making it impossible to prepare a condition that was bridged from the hub chain. Furthermore, all of the different operations like redeeming are impossible as they check that the condition is prepared:

```
uint256[] memory storedConditions = conditions[conditionId];  
require(storedConditions.length > 0, ConditionNotPrepared());
```

Also this can cause liquidity fragmentation where the same position has different representations across chains.

### Recommendations:

The easiest fix is to simply not include the `block.chainid` in the position's compute and instead use a placeholder number that is the same across all chains.



**Customer Response:**

Fixed in commit [ca67f74](#).

**Fix Review:**

Fix confirmed.



## Low Severity Issues

### L-01: Compressing will not work correctly in an edge case

|                                               |                |                 |
|-----------------------------------------------|----------------|-----------------|
| Severity: Low                                 | Impact: Medium | Likelihood: Low |
| Files:<br>src/modules/CombinatorialModule.sol | Status: Fixed  |                 |

#### Description:

The `compress` functionality purposefully reverts in the case where a leg has partially resolved so that the function is simpler and less prone to weird edge cases:

```
for (uint256 i; i < length;) {
    (bool resolved, uint256 conditionPayout) =
_getConditionPayout(storedConditions[i]);
    if (!resolved) {
        // ...
    } else if (conditionPayout == RESULT_DENOMINATOR) {
        // ...
    } else if (conditionPayout == 0) {
        anyFalse = true;
    } else {
        revert PositionNotCompressible();
    }

    if (anyFalse) break;
    unchecked {
        ++i;
    }
}
```

The issue is that the code behaves differently based on the order of the legs. For example, imagine a user holds `NO(A ^ B)` position. Imagine `A` resolves to `0.5` and `B` resolves to `NO`. In such a case, a user will not be able to use the `compress` functionality as we will revert on the iteration

over the **A** leg. However, imagine the user held  $\text{NO}(B \wedge A)$  (in reality, the code enforces ascending order of legs, this is just an example to paint a picture). In such a case, a user will be able to compress his position as we will enter the **conditionPayout == 0** case, set **anyFalse** as true and break out of the loop, then a user will get a payout for his position.

### Recommendations:

Consider handling the case mentioned. For example, in the **else** case, run a loop for the remaining legs and have the same **anyFalse** logic – then if the loop finishes there and **anyFalse** is still **false**, revert. Otherwise, let the user compress (technically redeem in this case) his position.

### Customer Response:

Fixed in commit [2959144](#).

### Fix Review:

Fix confirmed.

## L-02: Order dependence in `_getPositionPayout` prevents early settlement of combinatorial YES positions

| Severity: Low                                 | Impact: Low   | Likelihood: Low |
|-----------------------------------------------|---------------|-----------------|
| Files:<br>src/modules/CombinatorialModule.sol | Status: Fixed |                 |

### Description:

`_getPositionPayout` behaves differently depending on the order of the legs when holding a YES combinatorial position.

```
if (outcomeIndex == 0) {
    uint256 payoutFactor = PAYOUT_FACTOR_DENOMINATOR;
    for (uint256 i; i < length;) {
        (bool resolved, uint256 conditionPayout) =
            _getConditionPayout(storedConditions[i]);
        require(resolved, PositionNotRedeemable());
        if (conditionPayout == 0) return 0;
        if (conditionPayout != RESULT_DENOMINATOR) {
            payoutFactor = FixedPointMathLib.mulDiv(payoutFactor,
                conditionPayout, RESULT_DENOMINATOR);
        }
        unchecked {
            ++i;
        }
    }
}
```

```
}  
  
    return FixedPointMathLib.mulDiv(_amount, payoutFactor,  
PAYOUT_FACTOR_DENOMINATOR);  
}
```

The problem is that the path requires all of the conditions to be resolved even if we already have a resolved condition with a 0 payout.

For example, say that a user holds a position with 2 conditions. If the first one is resolved and the payout is 0, while the second one is unresolved, `_getPositionPayout` will return 0 and the redeem will correctly execute. However if the first one is unresolved and the second condition is resolved with a payout of 0 the redeem will revert because it will require the first condition to be resolved.

### **Recommendations:**

Consider doing the same thing as in the NO case because we would already know the outcome of the combinatorial position if one leg resolves to 0.

### **Customer Response:**

Fixed in commit [abedba3](#).

### **Fix Review:**

Fix confirmed.

## Informational Severity Issues

I-01: prepareCondition should use getConditionId to compute the conditionId

### Files:

src/modules/CombinatorialModule.sol

### Description:

In `CombinatorialModule.sol`, the contract defines a public view function `getConditionId` which is responsible for computing the unique `bytes32` condition Id for a given set of conditions. While all of the other functions directly use `getConditionId`, `prepareCondition` manually rewrites the logic instead of utilizing the already existing helper function.

### Recommendations:

Utilize the `getConditionId` helper function in `prepareCondition`.

### Customer Response:

We'll acknowledge this, `PositionIdLib.encodeConditionIdFromData` is only used in two places in the module, and `getConditionId` takes memory, but `prepareCondition` can encode directly from calldata. So it's not worth creating an internal function just for it.

## I-02: Array trimming functionality can be made into an $O(1)$ operation

### Files:

src/modules/CombinatorialModule.sol

### Description:

Trimming an array works as follows:

```
/// @dev Trims an oversized array to the given length.
function _trimArray(uint256[] memory _arr, uint256 _length) internal pure returns
(uint256[] memory) {
    uint256[] memory trimmed = new uint256[](_length);
    for (uint256 i; i < _length;) {
        trimmed[i] = _arr[i];
        unchecked {
            ++i;
        }
    }
    return trimmed;
}
```

It runs in  $O(N)$ , but can be made into an  $O(1)$  operation using the below code:

```
assembly {
    mstore(_arr, _length)
}
```

This works, because the first word of `_arr` is the length so the above code just rewrites it. The return array can be removed as the `_arr` input is modified in-place, then the main flow can simply use the input that was provided to the helper function. Note that this does not preserve the original which is fine in the current code, but it's an assumption that must hold in any future versions of the code.

### Recommendations:

Consider implementing the above logic.

### Customer Response:



Fixed in commit [e6f8192](#).

**Fix Review:**

Fix confirmed.

# Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

# About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.